

Information Retrieval

Text Analysis and Processing

Last lesson

- ❖ Boolean indexing and search
- ❖ Term-document matrix
- ❖ Inverted index
- ❖ Inverted index construction

Plan for this lecture

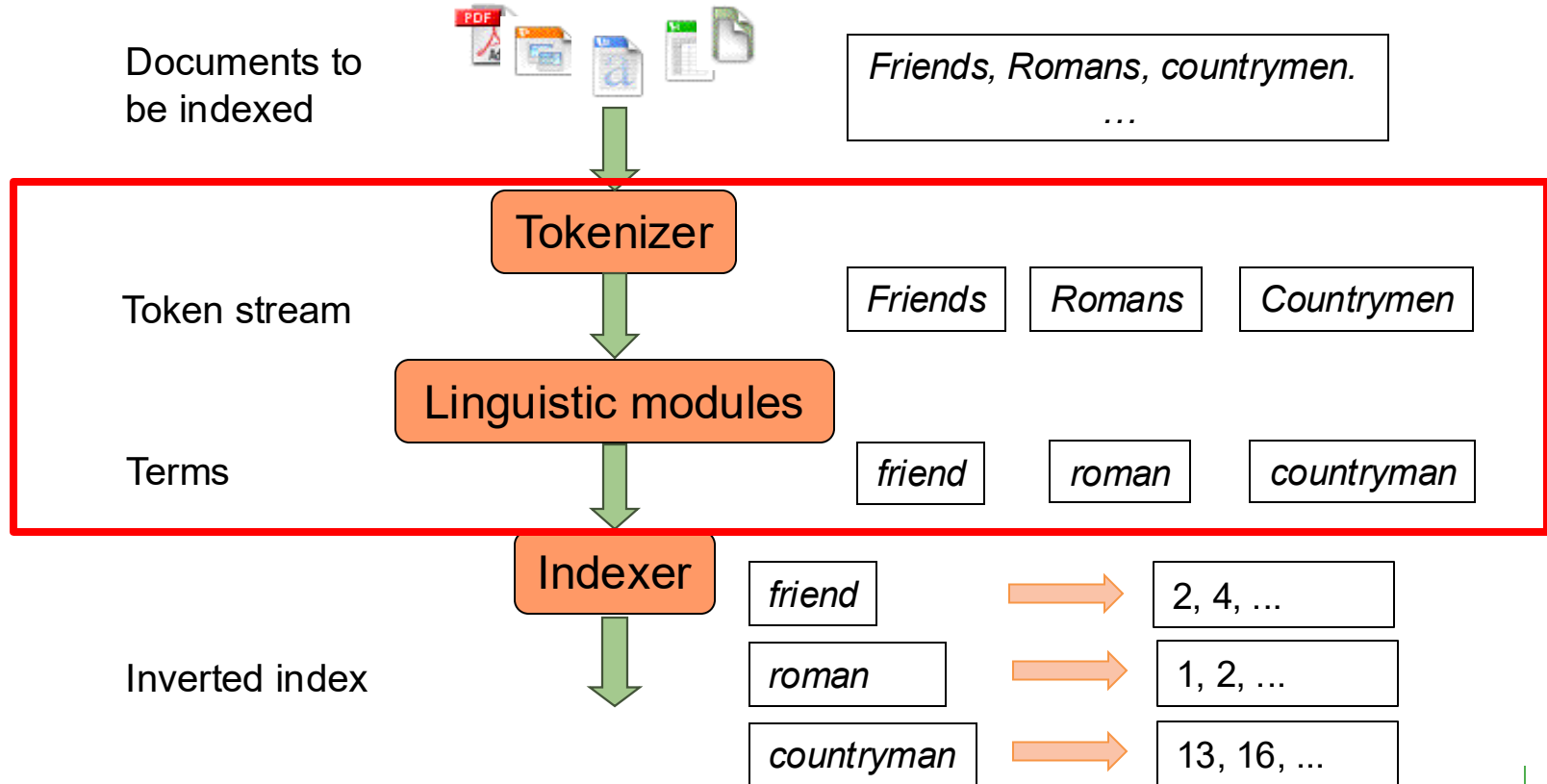
❖ Preprocessing to form the term vocabulary

- Tokenisation - classic
- Stopping
- Lemmatization
- Stemming

❖ Handling phrases

❖ Tokenisation - BPE

Inverted index construction



Preprocessing to form the vocabulary

Tokenization

- ❖ Input: “Friends, Romans and Countrymen”
- ❖ Output: Tokens
 - Friends
 - Romans
 - Countrymen
- ❖ A token is an instance of a sequence of characters
- ❖ Each such token is now a candidate for an index entry, after further processing
 - Described below
- ❖ But what are valid tokens to emit?

Tokenization

- ❖ Alphanumeric sequences with 3 or more characters
- ❖ Example:
 - “Bigcorp's 2007 bi-annual report showed profits rose 10%.”
- ❖ ... becomes
 - “bigcorp 2007 annual report showed profits rose”
- ❖ Too simple for search applications or even large-scale experiments
- ❖ Why?
 - Information lost
 - Small decisions in tokenizing can have a major impact on the effectiveness of some queries

Issues in tokenization

- ❖ Finland's capital →
 - Finland AND s? Finlands? Finland's?
- ❖ Hewlett-Packard → Hewlett and Packard as two tokens?
 - state-of-the-art: break up hyphenated sequence.
 - co-education
 - lowercase, lower-case, lower case ?
 - It can be effective in getting the user to put in possible hyphens
- ❖ San Francisco: one token or two?
 - How do you decide it is one token?

Issues with numbers

❖ Examples

- 3/20/91 Mar. 12, 1991 20/3/91
- 55 B.C.
- B-52
- My PGP key is 324a3df234cb23e
- (800) 234-2333

❖ Often have embedded spaces

❖ IR systems may not index numbers

- But often very useful
 - think about things like looking up error codes/stacktraces on the web
- (One answer is using n-grams: IIR ch. 3)

❖ Will often index “meta-data” separately

- Creation date, format, etc.

Stop words

- ❖ With a stop list, you exclude from the dictionary entirely the commonest words. Intuition:
 - They have little semantic content: the, a, and, to, be
 - There are a lot of them: ~30% of postings for the top 30 words
- ❖ But the trend is away from doing this:
 - Good compression techniques mean the space for including stop words in a system is small
 - Good query optimization techniques mean you pay little at query time for including stop words.
 - You need them for:
 - Phrase queries: “King of Denmark”
 - Various song titles, etc.: “Let it be”, “To be or not to be”
 - “Relational” queries: “flights to London”

Normalization to terms

- ❖ We may need to “normalize” words in indexed text as well as query words into the same form
 - We want to match U.S.A. and USA
- ❖ Result is terms:
 - a term is a (normalized) word type, which is an entry in our IR system dictionary
- ❖ We most commonly implicitly define equivalence classes of terms by, e.g.,
 - deleting periods to form a term
 - U.S.A., USA → USA
 - deleting hyphens to form a term
 - anti-discriminatory, antidiscriminatory → antidiscriminatory

Normalization: other languages

- ❖ Accents: e.g., French résumé vs. resume.
- ❖ Umlauts: e.g., German: Tuebingen vs. Tübingen
 - Should be equivalent
- ❖ Most important criterion:
 - How are your users like to write their queries for these words?
- ❖ Even in languages that standardly have accents, users often may not type them
 - Often best to normalize to a de-accented term
 - Tuebingen, Tübingen, Tubingen → Tübingen

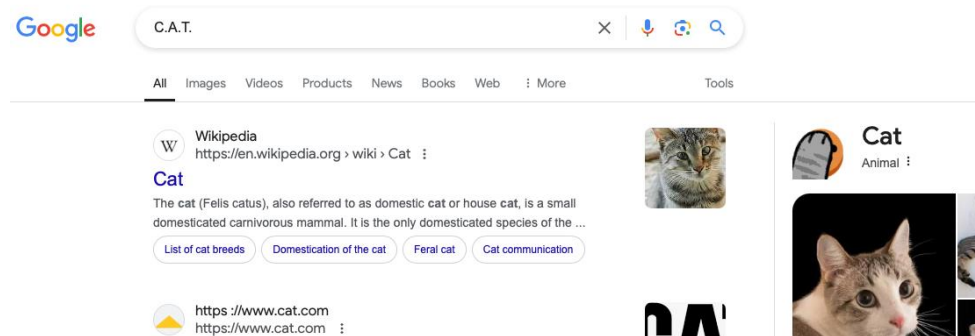
Case folding

❖ Reduce all letters to lower case

- exception: upper case in mid-sentence?
 - e.g., General Motors
 - Fed vs. fed
 - SAIL vs. sail
- Often best to lower case everything, since users will use lowercase regardless of ‘correct’ capitalization...

❖ Google example:

- Query: C.A.T.



Thesauri and soundex

❖ Do we handle synonyms and homonyms?

– E.g., by hand-constructed equivalence classes

- car = automobile color = colour

– We can rewrite to form equivalence-class terms

- When the document contains automobile, index it under car-automobile (and vice-versa)

– Or we can expand a query

- When the query contains automobile, look under car as well

❖ What about spelling mistakes?

– One approach is Soundex, which forms equivalence classes of words based on phonetic heuristics

Lemmatization

- ❖ Reduce inflectional/variant forms to base form
- ❖ E.g.,
 - am, are, is → be
 - car, cars, car's, cars' → car
- ❖ the boy's cars are different colors → the boy car be different color
- ❖ Lemmatization implies doing “proper” reduction to dictionary headword form

Stemming

- ❖ Many morphological variations of words
 - inflectional (plurals, tenses)
 - derivational (making verbs nouns etc.)
- ❖ In most cases, these have the same or very similar meanings
- ❖ Stemmers attempt to reduce morphological variations of words to a common stem
 - usually involves removing suffixes
- ❖ Can be done at indexing time or as part of query processing (like stopwords)

Stemming

❖ Two basic types

- Dictionary-based: uses lists of related words
- Algorithmic: uses program to determine related words

❖ Algorithmic stemmers

- suffix-s: remove 's' endings assuming plural
 - e.g., cats → cat, lakes → lake, wiis → wii
- Some problems:
 - supplies → supplie (should be supply)
 - ups → up (different concept)

❖ Some stemmers

- Porter
- Snowball
 - <http://snowball.tartarus.org>
- Lancaster stemmer

Porter's algorithm

- ❖ Common algorithm for stemming English
 - Results suggest it's at least as good as other stemming options
- ❖ Conventions + 5 phases of reductions
 - phases applied sequentially
 - each phase consists of a set of commands
 - select the one that applies to the longest suffix.
- ❖ Typical rules
 - ational → ate (e.g., rational → rate)
 - tional → tion (e.g., conventional → convention)
 - sses → ss (e.g., guesses → guess)
 - ies → i (e.g., dictionaries → dictionary)
 - (m>1) EMENT →
 - replacement → replac
 - cement → cement

Language-specificity

- ❖ The above methods embody transformations that are
 - Language-specific, and often
 - Application-specific
- ❖ English: very mixed results. It helps recall for some queries but harms the precision on others
 - E.g., operative (dentistry) $\Rightarrow$ oper
- ❖ Useful for Spanish, German, Finnish, ...

Phrase queries and positional indexes

Phrase queries

- ❖ Want to be able to answer queries such as “stanford university” – as a phrase
- ❖ Thus, the sentence “I went to university at Stanford” is not a match.
 - The concept of phrase queries has proven easily understood by users; one of the few “advanced search” ideas that works
 - Many more queries are implicit phrase queries
- ❖ For this, storing { term : [docs] } is not sufficient
 - Need more vocabulary’s entries, OR
 - The posting list structure must be expanded

Phrases

- ❖ Text processing issue – how are phrases recognized?
- ❖ Two possible approaches:
 - Use word n-grams (n-words)
 - Store word positions in indexes and use proximity operators in queries

Phrases - Biword indexes

- ❖ Index every consecutive pair of terms in the text as a phrase
- ❖ For example the text “Friends, Romans, Countrymen” would generate the biwords
 - friends romans
 - romans countrymen
- ❖ Each of these biwords is now a dictionary term
- ❖ Two-word phrase query-processing is now immediate.

Phrases - Biword indexes

- ❖ Longer phrases are processed by combining shorter phrases:
 - stanford university palo alto
- ❖ can be broken into the boolean query on biwords:
 - stanford university AND university palo AND palo alto
- ❖ Without the docs, we cannot verify that the docs matching the above Boolean query do contain the phrase.
 - we can have false positives!
- ❖ Index blowup due to bigger dictionary
 - Already big with biwords, infeasible for more than biwords
- ❖ Biword indexes are not the standard solution (for all biwords) but can be part of a compound strategy

Phrases - Positional indexes

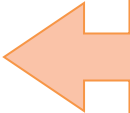
- ❖ In the postings, store, for each term the position(s) in it appears:
 - <term, number of docs containing term;
 - doc1: position1, position2 ... ;
 - doc2: position1, position2 ... ;
 - etc.>

- Note: to use the correct terminology, tokens appear in documents; the indexed form of a token is a term
 - Ex: “Operate an operating system...”
 - <oper; doc1: 0, 2>

Positional index example

- ❖ For phrase queries, we use a merge algorithm recursively at the document level
 - But we now need to deal with more than just equality

<**be**: 993427;
1: 7, 18, 33, 72, 86, 231;
2: 3, 149;
4: 17, 191, 291, 430, 434;
5: 363, 367, ...>



Which of docs **1,2,4,5**
could contain “*to be*
or not to be”?

Processing a phrase query

- ❖ Extract inverted index entries for each distinct term: to, be, or, not
- ❖ Merge their doc:position lists to enumerate all positions with “to be or not to be”
 - to:
 - 2:1,17,74,222,551;
 - 4:8,16,190,429,433;
 - 7:13,23,191; ...
 - be:
 - 1:17,19;
 - 4:17,191,291,430,434;
 - 5:14,19,101; ...
- ❖ Same general method for proximity searches

Positional index size

- ❖ Need an entry for each occurrence, not just once per document
- ❖ Index size depends on the average document size
 - Average web page has <1000 terms
 - Books, ... easily 100,000 terms
- ❖ Consider a term with frequency 0.1%

Document size	Postings	Positional postings
1000	1	1
100,000	1	100

- ❖ A positional index is 2–4 as large as a non-positional index
- ❖ Positional index size 35–50% of the volume of the original text

Tokenisation - BPE

Issues with rule-based tokenization

- ❖ Mostly but not always remove punctuation:
 - m.p.h., Ph.D., AT&T, cap'n
 - prices (\$45.55)
 - dates (01/02/06)
 - URLs (<http://www.stanford.edu>)
 - hashtags (#nlproc)
 - email addresses (someone@cs.colorado.edu)
- ❖ Numbers are tokenized differently across languages
 - English 555,500.50 = French 555 500,50
- ❖ Multiword expressions (MWE)?
 - New York, rock 'n' roll

Tokenization: language issues

❖ French

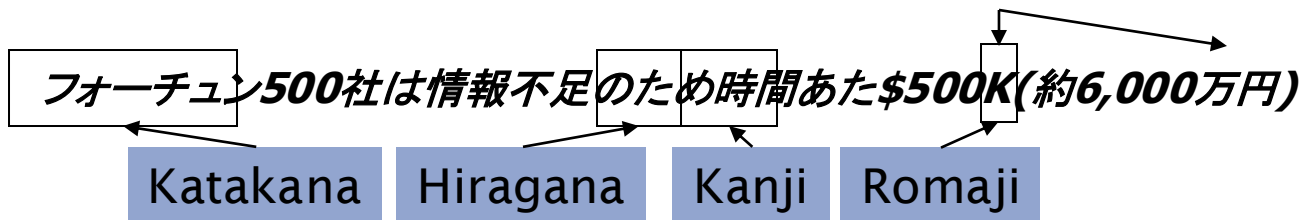
- L'ensemble → one token or two?
 - L ? L' ? Le ?
 - Want "l'ensemble" to match with "un ensemble"

❖ German noun compounds are not segmented

- Lebensversicherungsgesellschaftsangestellter
- 'life insurance company employee'
- German retrieval systems benefit greatly from a compound splitter module
 - Can give a 15% performance boost for German

Tokenization: language issues

- ❖ Chinese and Japanese have no spaces between words:
 - 莎拉波娃现在居住在美国东南部的佛罗里达。
 - Not always guaranteed a unique tokenization
- ❖ Further complicated in Japanese, with multiple alphabets intermingled
 - Dates/amounts in multiple formats



- End-user can express query entirely in hiragana!

Tokenization: language issues

- ❖ Arabic (or Hebrew) is written right to left, but with certain items like numbers written left to right
- ❖ Words are separated, but letter forms within a word form complex ligatures
 - ‘Algeria achieved its independence in 1962 after 132 years of French occupation.’

استقلت الجزائر في سنة 1962 بعد 132 عام من الاحتلال الفرنسي.
← start ← → ← →

- ❖ With Unicode, the surface presentation is complex, but the stored form is straightforward

Issue: ... there are simply too many words!

- ❖ Notice that (roughly) the bigger the corpora, the more words we find!

	Types = $ V $	Instances = N
Shakespeare	31 thousand	884,000
Brown Corpus	38 thousand	1 million
Switchboard conversations	20 thousand	2.4 million
COCA	2 million	440 million
Google N-grams	13+ million	1 trillion

There are simply too many words!

- ❖ N = number of instances
- ❖ $|V|$ = number of types in vocabulary V
- ❖ Heaps Law = Herdan's Law

$$|V| = kN^{\beta} \leftarrow \text{Roughly } 0.5$$

- ❖ Vocab size for a text goes up with the square root of its length in words

Types of Tokenisation

❖ Whole-Word Tokenisation

- Based on spaces and punctuation. It's the simplest approach, but it's ineffective for unknown words.
- No matter how big our vocabulary, there will always be words we miss!

❖ Character Tokenisation

- Every character is a token. This creates a very large vocabulary and can lead to a loss of context of words.

❖ Subword Tokenisation

- Modern NLP systems don't use words, but smaller units called subwords
- Optimises the vocabulary by breaking down words into smaller, reusable parts (e.g., using the Byte-Pair Encoding - BPE algorithm).
- The idea is to use the data to tell us how to tokenise.

Subword tokenization

- ❖ Two most common algorithms:
 - Byte-Pair Encoding (BPE) (Sennrich et al., 2016)
 - Unigram language modeling tokenization (Kudo, 2018) (sometimes confusingly called "SentencePiece" after the library it's in)
- ❖ All have 2 parts:
 - ❖ **A token learner**
 - that takes a raw training corpus and induces a vocabulary (a set of tokens).
 - ❖ **A token segmenter**
 - that takes a raw test sentence and tokenizes it according to that vocabulary

How Byte-Pair Encoding (BPE) Works

- ❖ The BPE algorithm builds the subword vocabulary through an iterative process.
 - Start: It begins with a vocabulary containing all unique characters from the dataset.
 - Merge Pairs: It repeatedly finds the most frequent pair of subwords and merges them into a new subword. This process is repeated until a pre-defined vocabulary size is reached.
- ❖ Example
 - The word "tokenisation" might be split into ['token', 'isation'] if those subwords are in the vocabulary.
 - To tokenise new text, the system uses the created merge rules to break down words into known vocabulary subwords, ensuring that even new words can be processed.

BPE token learner

❖ Iteratively merge frequent neighbouring tokens to create longer tokens.

❖ Repeat:

- Choose the most frequent neighbouring pair ('A', 'B')
- Add a new merged symbol ('AB') to the vocabulary
- Replace every 'A' 'B' in the corpus with 'AB'.

❖ Until k merges

❖ Don't merge across word boundaries

- First, separate corpus by whitespace
- This gives a set of starting strings, with whitespace attached to the front of them
- Counts come from the corpus, but can only merge within strings.

Vocabulary

[A, B, C, D, E]

[A, B, C, D, E, AB]

[A, B, C, D, E, AB, CAB]

Corpus

A B D C A B E C A B

AB D C AB E C AB

AB D CAB E CAB

BPE token learner algorithm

function BYTE-PAIR ENCODING(strings C , number of merges k) **returns** vocab V

$V \leftarrow$ all unique characters in C # initial set of tokens is characters

for $i = 1$ **to** k **do** # merge tokens til k times

$t_L, t_R \leftarrow$ Most frequent pair of adjacent tokens in C

$t_{NEW} \leftarrow t_L + t_R$ # make new token by concatenating

$V \leftarrow V + t_{NEW}$ # update the vocabulary

 Replace each occurrence of t_L, t_R in C with t_{NEW} # and update the corpus

return V

BPE token learner

- ❖ Original (very simple) corpus:
 - `set_new_new_renew_reset_renew`
- ❖ Put space token at start of words

corpus

2 _ n e w

2 _ r e n e w

1 s e t

1 _ r e s e t

vocabulary

_, e, n, r, s, t, w

BPE token learner

corpus

```
2  _ n e w
2  _ r e n e w
1  s e t
1  _ r e s e t
```

vocabulary

_, e, n, r, s, t, w

Merge **n** **e** to **ne** (count 4 = 2 **new** + 2 **renew**)

corpus

```
2  _ ne w
2  _ r e ne w
1  s e t
1  _ r e s e t
```

vocabulary

_, e, n, r, s, t, w, ne

BPE token learner

corpus

2 _ ne w

2 _ r e ne w

1 s e t

1 _ r e s e t

vocabulary

_, e, n, r, s, t, w, ne

Merge **ne w** to **new** (count 4)

corpus

2 _ new

2 _ r e new

1 s e t

1 _ r e s e t

vocabulary

_, e, n, r, s, t, w, ne, new

BPE token learner

corpus

2 _ new

2 _ r e new

1 s e t

1 _ r e s e t

vocabulary

_, e, n, r, s, t, w, ne, new

Merge _ r to _r (count 4) and _r e to _re (count 3)

corpus

2 _ new

2 _re new

1 s e t

1 _re s e t

vocabulary

_, e, n, r, s, t, w, ne, new, _r, _re

BPE

The next merges are:

merge	current vocabulary
(-, new)	-, e, n, r, s, t, w, ne, new, -r, -re, -new
(-re, new)	-, e, n, r, s, t, w, ne, new, -r, -re, -new, -renew
(s, e)	-, e, n, r, s, t, w, ne, new, -r, -re, -new, -renew, se
(se, t)	-, e, n, r, s, t, w, ne, new, -r, -re, -new, -renew, se, set

BPE – another example

❖ Original:

- 'South Australia (SA) has a unique position in Australia's history as, unlike the other states which were founded as colonies, South Australia began as a self governing province Many were attracted to this and Adelaide and SA developed as an independent and free thinking state.'

❖ BPE token segmenter:

- ['south_', 'austr', 'ali', 'a_', '(_', 's', 'a_', ')_', 'has_', 'a_', 'uni', 'qu', 'e_', 'posi', 'tion_', 'in_', 'austr', 'ali', 'a_', '"', 's_', 'histor', 'y_', 'as_', '_, 'un', 'li', 'ke_', 'the_', 'other_', 'states_', 'which_', 'were_', 'foun', 'ded_', 'as_', 'col', 'on', 'ies_', '_, 'south_', 'austr', 'ali', 'a_', 'beg', 'an_', 'as_', 'a_', 'sel', 'f_', 'gover', 'ning_', 'pro', 'v', 'in', 'ce_', 'many_', 'were_', 'at', 'tr', 'ac', 'ted_', 'to_', 'this_', 'and_', 'ad', 'el', 'ai', 'de_', 'and_', 's', 'a_', 'develop', 'ed_', 'as_', 'an_', 'in', 'dep', 'end', 'ent_', 'and_', 'f', 'ree_', 'th', 'in', 'king_', 'state_', '._']

BPE Advantages

❖ Vocabulary Management

- Modern language models deal with enormous vocabularies. Subword tokenisation helps keep the vocabulary more compact and efficient.

❖ Handling Out-of-Vocabulary (OOV) Words

- A system based on whole words fails when it encounters a new term or a typo. BPE solves this by breaking the word into sub-parts that already exist in the vocabulary. For example, the word "unbelievable" can be split into ['un', 'believe', 'able'].

❖ Capturing Morphology

- Subwords automatically capture the morphological structure of a language (prefixes, suffixes, etc.), allowing the model to generalise knowledge from one word to others. For instance, "running" and "eating" might share the subword "ing".

BPE Advantages

- ❖ Loss of semantic precision
 - A search for ['car', 'red'] may be more precise than one for ['car', 'ro', 'red']. Segmentation into subwords can fragment the meaning of a word.
- ❖ Complexity in search logic
 - The logic for combining and scoring subword results can be more complex than simply using whole words.
- ❖ In modern search systems, such as those of large technology companies, the use of an inverted index with BPE is just one part of a hybrid system that also uses semantic search techniques, such as vector embeddings, to complement keyword search.
 - Further details will be provided later in this course.

This lesson

- ❖ Documents
- ❖ Tokenization
- ❖ Stopping
- ❖ Lemmatization
- ❖ Stemming
- ❖ Handling phrases: bigram index, positional index
- ❖ BPE